

A Neural Particle-in-Cell Architecture for High-Fidelity Deformable Physics Simulation

Phillips Li Zhao Xu Shiyuan Zhang Tong Xie

Department of Computer Science, University of California, Los Angeles
CS 269: Deformable Models, Spring 2026

Abstract

Learned simulators promise orders-of-magnitude speedups over classical numerical solvers for deformable physics, but their long-horizon stability remains a central obstacle. We propose *Neural PIC*, a hierarchical graph network with a U-Net topology whose pooling, coarse-level processing, and unpooling are designed to mirror the particle-to-grid transfer, grid solve, and grid-to-particle transfer of the classical Particle-in-Cell (PIC) scheme: message passing on a proximity graph resolves local kinematics, Self-Attention Graph (SAG) pooling coarsens the graph toward a low-resolution proxy, a Transformer resolves global dynamics at the coarsest level, and skip-connected unpooling returns the resolved signal to the full particle set. The design hypothesis is that the particle→grid information bottleneck that stabilizes classical PIC will transfer to the neural setting, simultaneously improving long-horizon accuracy (H1) and reducing the cost of global attention (H2). We test both hypotheses against Graph Network-based Simulators (GNS) on the Goop deformable benchmark over full 395-step autoregressive rollouts, measuring position MSE, kinetic-energy error, Sinkhorn distance, and per-pass efficiency. **Both hypotheses are rejected.** GNS is more accurate on every metric—by roughly $4.5\times$ in mean position MSE and up to $19\times$ in mean kinetic-energy error—and our model wins on only 2 of 30 test trajectories, with the gap widening over the rollout (the median per-trajectory error ratio grows from 4.58 overall to 6.87 over the final 40 steps). On efficiency, our model is 30% smaller in parameters yet $\sim 64\%$ slower per forward pass. We report this as a negative result and discuss what it implies about transcribing the PIC stability mechanism into a learned, attention-bottlenecked architecture.

1 Introduction

High-fidelity simulation of deformable materials underpins applications across real-time graphics, engineering design, and robotic manipulation. The classical approach—discretizing the governing partial differential equations and integrating them forward in time—is accurate but computationally expensive, and this cost is a hard bottleneck for any interactive or iterative workflow. Learned simulators offer an attractive alternative: amortize the cost of physics into a neural network that predicts the next state directly, trading a one-time training cost for cheap inference.

The dominant paradigm for particle-based learned simulation is the Graph Network-based Simulator (GNS) [7], which applies several rounds of flat message passing over a proximity graph of particles. GNS is simple, strong, and widely adopted. Its principal weakness is structural: message passing propagates information only locally per step, so capturing global material constraints requires many rounds, and long-horizon rollouts are prone to compounding error.

We ask whether *architectural* structure borrowed from classical computational physics can address this. The Particle-in-Cell (PIC) method couples moving Lagrangian particles with a fixed Eulerian background grid: particle quantities are scattered to the grid, global dynamics are resolved on the grid, and the result is gathered back to particles. The grid acts as a low-dimensional bottleneck through which global information must pass, and the fixed-grid scatter/gather is numerically exact. We hypothesize that an analogous learned bottleneck could (H1) stabilize long-horizon rollouts and (H2) make global reasoning cheaper than dense particle-level computation.

We instantiate this idea as **Neural PIC**, a Graph-U-Net whose pooling and unpooling play the role of PIC’s particle→grid transfers and whose coarse-level self-attention plays the role of the global grid solve, and evaluate it against GNS on the Goop benchmark as a clean, reproducible test of whether the PIC stability mechanism survives translation into a learned form. We find that it does not: on Goop the model is less accurate and slower than the GNS baseline, and the accuracy gap grows with rollout length rather than shrinking.

Our contributions are:

- A hierarchical architecture (Neural PIC) that casts each stage of a Graph-U-Net as an analogue of classical PIC: message passing as particle-to-grid transfer, SAG pooling as grid coarsening, coarse-level self-attention as the global grid solve, and skip-connected unpooling as grid-to-particle transfer.
- A controlled evaluation against GNS on full Goop rollouts, measuring position MSE, kinetic-energy error, Sinkhorn distance, and per-pass efficiency.
- An analysis of when and why this learned particle→grid bottleneck fails to inherit classical PIC’s stability and efficiency on this benchmark, with concrete conditions under which it might be expected to win.

2 Related Work

Neural PDE solvers. Physics-Informed Neural Networks (PINNs) [6] embed PDE residuals into the training objective, solving forward and inverse problems without explicit discretization, and neural operators learn solution maps between function spaces. These methods are largely Eulerian and grid-

based, and are most natural for problems where the domain is fixed.

Learned particle simulators. For Lagrangian systems with large deformation and topology change, particle representations are preferred. GNS [7] is the canonical learned particle simulator: it builds a radius graph over particles and applies an encode–process–decode stack of message-passing layers, predicting per-particle accelerations. Its flat structure makes it a strong baseline but limits the range over which information propagates per step. Graph U-Nets [4] introduce learnable pooling and unpooling to give graph networks a multi-resolution receptive field; SAG pooling [5] scores nodes with a self-attention layer and retains a top fraction. Our work asks whether such a hierarchical, PIC-inspired structure improves on flat message passing for deformable rollouts.

Benchmarks. We evaluate on Goop, a viscous deformable material from the standard learned-simulation suite, and refer to FD-Bench [9] for broader context on fair, modular comparison of data-driven simulators.

3 Problem Statement

We model a deformable body as a set of N interacting particles obtained by Lagrangian discretization of a continuous medium. The kinematic state at time t is the coordinate matrix $\mathbf{X}^t \in \mathbb{R}^{N \times d}$ stacking all particle positions ($d=2$ for Goop). The task is to predict future states from past ones.

We frame this as autoregressive sequence-to-sequence prediction. Given a temporal context window of H historical frames $\mathbf{X}^{t-H+1}, \dots, \mathbf{X}^t$, the model learns a forward-dynamics map f_θ that predicts the next frame,

$$\hat{\mathbf{X}}^{t+1} = f_\theta(\mathbf{X}^{t-H+1}, \dots, \mathbf{X}^t), \quad (1)$$

and is rolled out by feeding predictions back as input. At evaluation we unroll f_θ for the full trajectory; with $H=6$ and 401-frame trajectories this yields 395 autoregressive steps. Because each step consumes its own predictions, errors can compound, making long-horizon stability the key property under test.

4 Method

We model the forward-dynamics map f_θ as a hierarchical graph neural network with a U-Net topology and self-attention at its coarsest level. At each step the particle set is endowed with a graph structure $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ indexes the N particles and $\mathcal{E}$ collects directed edges (i, j) between spatially proximate particles. Node features encode per-particle kinematic state aggregated over the history window and edge features encode relative geometry; we write $\mathbf{v}_i^{(0)} \in \mathbb{R}^{F_v}$ and $\mathbf{e}_{ij}^{(0)} \in \mathbb{R}^{F_e}$ for the raw inputs.

4.1 Encoder

Two multilayer perceptrons lift nodes and edges into a shared latent space of width D ,

$$\mathbf{h}_i = \phi_v^{\text{enc}}(\mathbf{v}_i^{(0)}), \quad \boldsymbol{\varepsilon}_{ij} = \phi_e^{\text{enc}}(\mathbf{e}_{ij}^{(0)}), \quad \mathbf{h}_i, \boldsymbol{\varepsilon}_{ij} \in \mathbb{R}^D, \quad (2)$$

each ϕ^{enc} being a MLP with ReLU activations and a terminal LayerNorm.

4.2 Message passing as particle–grid transfer

The core computational primitive is a residual graph network block that we apply repeatedly at every resolution. A single block updates edges and then nodes:

$$\boldsymbol{\varepsilon}'_{ij} = \boldsymbol{\varepsilon}_{ij} + \psi_e([\mathbf{h}_i \| \mathbf{h}_j \| \boldsymbol{\varepsilon}_{ij}]), \quad (3)$$

$$\mathbf{m}_j = \sum_{i \in \mathcal{N}(j)} \boldsymbol{\varepsilon}'_{ij}, \quad (4)$$

$$\mathbf{h}'_j = \mathbf{h}_j + \psi_v([\mathbf{h}_j \| \mathbf{m}_j]), \quad (5)$$

where $[\cdot \| \cdot]$ denotes concatenation, $\mathcal{N}(j) = \{i : (i, j) \in \mathcal{E}\}$ is the set of senders into j , and ψ_e, ψ_v are MLPs with the same architecture as the encoder (without terminal normalization). The residual connections on both edges and nodes follow the Encode–Process–Decode paradigm and stabilize deep stacks of blocks.

Equations (3)–(5) are structurally identical to the particle-to-grid (P2G) phase of the Particle-in-Cell (PIC) method. In PIC, a field quantity carried by particle i is transferred to a grid node j through a transfer weight w_{ij} that depends on the relative position of the pair, and the grid value is formed by summing all incident particle contributions,

$$q_j = \sum_i w_{ij} q_i. \quad (6)$$

Our learned message $\psi_e(\cdot)$ in (3) plays the role of the transfer kernel $w_{ij} q_i$: rather than a fixed interpolation stencil it is a data-driven function of the sender state $\mathbf{h}_i$, the receiver state $\mathbf{h}_j$, and the edge geometry $\boldsymbol{\varepsilon}_{ij}$, the last of which generalizes the relative-position dependence of w_{ij} . The permutation-invariant sum in (4) is exactly the accumulation in (6): each receiver gathers and scatters contributions from all of its neighbors into a single pooled quantity $\mathbf{m}_j$. Concretely, (4) is realized as a scatter-add of edge features indexed by receiver, the discrete analogue of depositing particle mass and momentum onto the background grid. The subsequent node update (5) then plays the role of the grid-level solve in PIC, mapping the accumulated deposit to an updated nodal state before it is transferred back. We exploit this correspondence at every level of the hierarchy described next, so that message passing on the fine graph mirrors deposition onto a fine grid, and message passing on coarsened graphs mirrors deposition onto progressively coarser grids.

4.3 Hierarchical encoder–decoder

To capture interactions across length scales we embed the message-passing block in a U-Net with L resolution levels.

We denote by $\ell = 0$ the finest level and process the graph top-down. At each downward level ℓ we first apply $S_\downarrow$ processor blocks (3)–(5), cache the result as a skip state, and then coarsen the graph:

$$\mathbf{H}^{(\ell)} = \left(\prod_{s=1}^{S_\downarrow} \text{GNBlock} \right) (\mathbf{H}^{(\ell)}, \mathcal{E}^{(\ell)}), \quad (7)$$

$$\mathbf{S}^{(\ell)} \leftarrow \mathbf{H}^{(\ell)} \quad (\text{skip connection}), \quad (8)$$

$$(\mathbf{H}^{(\ell+1)}, \mathcal{E}^{(\ell+1)}, \boldsymbol{\pi}^{(\ell)}) = \text{Pool}_r(\mathbf{H}^{(\ell)}, \mathcal{E}^{(\ell)}). \quad (9)$$

The pooling operator is self-attention graph pooling, which scores every node, retains the top fraction $r \in (0, 1]$ of nodes, and records the retained indices $\boldsymbol{\pi}^{(\ell)}$. This selects a representative subset of particles to serve as the coarse grid at the next level, the learned counterpart of choosing a coarser cell spacing in a multigrid PIC solver.

At the coarsest level the graph is small enough to permit dense, all-to-all interaction. We therefore replace local message passing with B self-attention blocks. Within each block, nodes belonging to the same trajectory in the batch are gathered into a dense tensor and processed by multi-head self-attention with a padding mask, followed by a position-wise MLP, each wrapped in a residual connection and LayerNorm:

$$\mathbf{Z} = \text{LayerNorm}(\mathbf{H}^{(L)} + \text{MHA}(\mathbf{H}^{(L)}, \mathbf{H}^{(L)}, \mathbf{H}^{(L)})), \quad (10)$$

$$\mathbf{H}^{(L)} = \text{LayerNorm}(\mathbf{Z} + \text{MLP}(\mathbf{Z})). \quad (11)$$

This bottleneck lets every coarse particle exchange information with every other, approximating the long-range coupling that a global field solve would provide in PIC while operating on the reduced node set.

The upward pass inverts the pooling. At each level ℓ (processed from coarse to fine) the coarse latents are scattered back onto the fine node set at the indices that were retained during pooling, the remaining nodes are initialized to zero, and the cached skip state is added back:

$$\tilde{\mathbf{H}}_{\boldsymbol{\pi}^{(\ell)}}^{(\ell)} = \mathbf{H}^{(\ell+1)}, \quad \tilde{\mathbf{H}}_{k \notin \boldsymbol{\pi}^{(\ell)}}^{(\ell)} = \mathbf{0}, \quad \mathbf{H}^{(\ell)} = \tilde{\mathbf{H}}^{(\ell)} + \mathbf{S}^{(\ell)}. \quad (12)$$

This unpooling is the grid-to-particle (G2P) counterpart of the transfer in (4.2): information resolved on the coarse grid is interpolated back to the finer particle set, and the additive skip connection preserves the high-frequency detail that pooling discarded. We then apply $S_\uparrow$ further processor blocks at the fine resolution to reconcile the transferred coarse signal with the retained fine features.

4.4 Decoder and rollout

A final MLP without output normalization maps the finest-level node latents to the per-particle prediction targets,

$$\hat{\mathbf{y}}_i = \phi^{\text{dec}}(\mathbf{h}_i^{(0)}) \in \mathbb{R}^d, \quad (13)$$

from which the next-frame state $\hat{\mathbf{X}}^{t+1}$ is reconstructed. Composing the encoder, the hierarchical processor, and the decoder yields the one-step map f_θ , which is unrolled autoregressively as described in the problem statement.

5 Experimental Setup

Dataset. We evaluate on the GOOP split of the deformable-materials benchmark introduced with the Graph Network Simulator (GNS) [7]. Each trajectory is a Lagrangian discretisation of a viscoplastic medium into a variable number of particles (297–1601 in the test split; mean 879). We use all 30 held-out test trajectories, each consisting of $T=401$ frames at $\Delta t=2.5 \times 10^{-3}$ in a $[0.1, 0.9]^2$ simulation domain.

Task. Given a context window of $H=6$ historical frames $\mathcal{X}_{t-H+1:t} \in \mathbb{R}^{H \times N_i \times 2}$ for the i -th trajectory with N_i particles, the learnt operator f_θ predicts the next-step position $\hat{\mathbf{X}}^{t+1} = f_\theta(\mathcal{X}_{t-H+1:t})$. Internally the model emits a per-particle velocity in the normalised frame; the prediction is unnormalised, added to the current positions, and clamped to the domain. The predicted frame replaces the ground-truth one in the context window, giving us an autoregressive rollout over $T-H=395$ steps per trajectory.

Metrics. We track three complementary error signals on every rollout step. Let $\mathbf{Y}, \hat{\mathbf{Y}} \in \mathbb{R}^{N_i \times 2}$ be the ground-truth and predicted positions, and $\mathbf{v}, \hat{\mathbf{v}}$ the corresponding one-step velocities $\mathbf{v} = \mathbf{Y} - \mathbf{X}^t$.

- **Position MSE** captures geometric accuracy:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2N_i} \sum_{p=1}^{N_i} \|\hat{\mathbf{Y}}_p - \mathbf{Y}_p\|_2^2. \quad (14)$$

- **Kinetic-energy error** measures physical consistency:

$$\mathcal{L}_{\text{KE}} = \frac{1}{N_i} \sum_{p=1}^{N_i} \left(\frac{1}{2} \|\hat{\mathbf{v}}_p\|_2^2 - \frac{1}{2} \|\mathbf{v}_p\|_2^2 \right)^2. \quad (15)$$

- **Debiased Sinkhorn divergence** [1, 2] treats the two particle clouds as discrete uniform measures $\mu, \hat{\mu}$ and reports $S_\varepsilon(\hat{\mu}, \mu) = \text{OT}_\varepsilon(\hat{\mu}, \mu) - \frac{1}{2}(\text{OT}_\varepsilon(\hat{\mu}, \hat{\mu}) + \text{OT}_\varepsilon(\mu, \mu))$, where OT_ε is the entropic optimal-transport cost under squared-Euclidean ground cost, computed by the Sinkhorn algorithm in POT [3] with $\varepsilon=0.1$, up to 500 inner iterations and tolerance 10^{-5} .

Per-trajectory means and final-step values are aggregated across the 30 trajectories by an unweighted mean and reported with the trajectory standard error.

Baseline and proposed model. The baseline is GNS [7] with $K=10$ identical message-passing blocks of width $d=128$. Our model is a Graph-U-Net [4] with $L=2$ pool/unpool levels and SAG pooling [5] at ratio 0.6, capped by six Transformer self-attention blocks [8] at the bottleneck. Both models share the same input features: current position, the past five normalised velocities, four signed wall distances, and a 3-class one-hot of the particle type. Edges are constructed with connectivity radius $r=0.015$ at every rollout step. We use the checkpoints supplied with the project repository without any further finetuning.

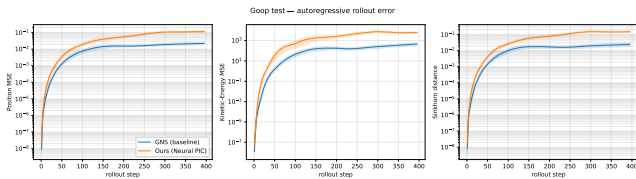


Figure 1. Per-step error on the Goop test split. Mean across 30 trajectories with shaded ± 1 standard error. Both models start from the same noise floor; the proposed model accelerates upward earlier and converges to a $4\times\text{--}5\times$ higher plateau on all three metrics.

Implementation. All experiments use PyTorch 2.5.1+cu124 with torch_geometric 2.7 and torch_scatter 2.1.2 on a single NVIDIA RTX A6000 (49 GB). Each model takes roughly 40 min of wall time to roll out the full test split.

6 Results

6.1 Long-horizon prediction quality

Table 1 reports the per-step average and the final-step value (step 395) of each metric, aggregated over the 30 test trajectories. Across every metric and every aggregation level, the GNS baseline outperforms our model by between $4.5\times$ (mean MSE) and $19\times$ (mean KE error).

The disparity is not concentrated on a few outlier trajectories. On the per-trajectory mean Position MSE, our model is better on 2 of 30 trajectories; the median ratio between the two models is 4.58, and on the last 40 rollout steps it widens further to 6.87. This pattern points to long-horizon stability as the discriminator, not local fit: Figure 1 shows the per-step mean and standard error across trajectories on a logarithmic vertical axis. Both models start in the same noise floor ($10^{-8}\text{--}10^{-7}$) on step 0, but the proposed model accelerates upward earlier and saturates at a higher plateau on every metric.

6.2 Compute efficiency

Table 2 reports the parameter count, single-pass latency and peak GPU memory for both models on a representative test trajectory (799 particles, $\sim 6\,300$ edges). Latency is the mean of 100 timed forward passes after 20 warm-up passes, with torch.cuda.synchronize barriers on either side; peak memory is torch.cuda.max_memory_allocated during a single forward pass.

Our model is 30% smaller in parameter count but $\sim 64\%$ slower per forward pass. The slowdown is dominated by the SAG-pool / Transformer pipeline: the additional graph rewiring, score-then-rank node selection, and the multi-head attention block together cost more wall-clock time than is saved by routing the costly message-passing operations through a thinner graph.

7 Discussion

Long-horizon stability is the discriminator. The two models are close at the start of every rollout—both sit in the same $10^{-8}\text{--}10^{-7}$ noise floor at step 0—and diverge only as the rollout proceeds (Sec. 6.1, Fig. 1). Our model accelerates upward

earlier and plateaus higher on all three metrics, and the per-trajectory error ratio against GNS grows from 4.58 overall to 6.87 over the final 40 steps. The failure is therefore not one of local one-step fit but of stability under autoregressive composition: small per-step biases accumulate faster in our model than in flat GNS.

The hierarchical bottleneck did not buy stability here.

The design hypothesis (H1) was that routing global information through a coarse, PIC-like bottleneck would regularize long rollouts. On Goop it did the opposite. Two features of the learned bottleneck distinguish it from the classical mechanism it imitates. First, the coarse grid is *content-dependent*: SAG pooling re-scores and re-selects which particles survive to the coarse level at every step, so the effective discretization shifts from frame to frame, whereas a classical PIC grid is fixed. Second, the coarse solve is a learned self-attention block rather than an exact field solve. The additive skip connections in (12) are explicitly intended to return the high-frequency detail that pooling drops, so the divergence is better read not as information simply being thrown away, but as the coarse pathway injecting a learned signal that does not stay consistent with the retained fine detail as the state drifts autoregressively away from the training distribution. Diagnosing which of these—moving coarse support versus the learned coarse solve—dominates is the natural next experiment (e.g. freezing the pooled index set across a rollout, or ablating the attention bottleneck).

Why the smaller model is slower. Our model has 30% fewer parameters yet is $\sim 64\%$ slower per forward pass (Sec. 6.2), so H2 is also rejected at this problem size. As the results note, the overhead is dominated by the SAG-pool/Transformer pipeline: graph rewiring, score-then-rank node selection, and the attention block cost more wall-clock time than is saved by shrinking the graph the message-passing layers run on. Parameter count is a poor proxy for latency here; the savings a coarse bottleneck offers are asymptotic in graph size, and Goop’s graphs (mean 879 particles) are far below the regime where dense particle-level computation would dominate.

When might a bottleneck win? The negative result is specific to this regime, and several untested regimes could change it. (i) *Larger graphs*, where flat message passing or dense particle-level computation is itself the bottleneck and the asymptotic savings of coarsening would actually be realized. (ii) *Materials with genuine long-range coupling*—elastic solids or shells—where local message passing is structurally insufficient and a global coarse solve is necessary; Goop’s viscous, largely local dynamics may already be well captured by flat GNS. (iii) *Alternative pooling*: a higher retention ratio, top- k instead of SAG, or pooling held fixed across a rollout to remove the moving-support effect discussed above. Evaluating the sand and water splits would test whether the failure is material-specific.

Table 1. Quantitative evaluation on the Goop test split (30×395 rollout steps). Lower is better. Bold marks the better model on each column.

Model	Position MSE		Kinetic-energy MSE		Sinkhorn	
	Mean	Final	Mean	Final	Mean	Final
GNS	$1.30 \cdot 10^{-2}$	$2.20 \cdot 10^{-2}$	$1.69 \cdot 10^2$	$4.54 \cdot 10^2$	$1.45 \cdot 10^{-2}$	$2.44 \cdot 10^{-2}$
Ours	$5.81 \cdot 10^{-2}$	$1.14 \cdot 10^{-1}$	$3.26 \cdot 10^3$	$6.23 \cdot 10^3$	$8.05 \cdot 10^{-2}$	$1.48 \cdot 10^{-1}$

Table 2. Compute efficiency for a single forward pass.

Model	Params (M)	Latency (ms)	Peak GPU memory (MiB)
GNS	1.970	6.27	45.4
Ours	1.378	10.31	45.7

Limitations. We evaluated the checkpoints supplied with the project repository without further finetuning, and did not verify that training had converged, so undertraining cannot be fully excluded as a confound. Edges are rebuilt at radius $r=0.015$ every step, and both models share input features and the same evaluation pipeline, but absolute numbers could shift under different normalization or connectivity choices. Finally, only the Goop split was evaluated; we make no claim about other materials.

8 Conclusion

We proposed Neural PIC, a Graph-U-Net whose pooling, coarse-level self-attention, and skip-connected unpooling are cast as learned analogues of the classical Particle-in-Cell particle→grid solve. On the Goop benchmark it underperforms the flat GNS baseline on both axes it was meant to improve: it is less accurate on every metric, with the gap widening over the rollout, and it is slower per forward pass despite having fewer parameters. The evidence suggests that a content-dependent, learned coarse bottleneck does not, on this regime, inherit the long-horizon stability that PIC’s fixed-grid, exact transfers provide, and that its efficiency advantage is asymptotic in a graph-size regime Goop does not reach. We frame this as a negative result that sharpens the open question: under what graph sizes, material types, and pooling schemes does a learned particle–grid bottleneck recover the stability and efficiency classical PIC enjoys by construction?

References

- [1] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [2] Jean Feydy, Thibault S  jour  n  , Fran  ois-Xavier Vialard, Shun-ichi Amari, Alain Trouv  , and Gabriel Peyr  . Interpolating between optimal transport and MMD using Sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2019.
- [3] R  mi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aur  lie Boissunon, Stanislas Cham-
- bon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, L  o Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021.
- [4] Hongyang Gao and Shuiwang Ji. Graph U-nets. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- [5] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. Self-attention graph pooling. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- [6] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- [7] Alvaro S  nchez-Gonz  lez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] Haixin Wang, Ruoyan Li, Fred Xu, Fang Sun, Kaiqiao Han, Zijie Huang, Guancheng Wan, Ching Chang, Xiao Luo, Wei Wang, and Yizhou Sun. FD-Bench: A modular and fair benchmark for data-driven fluid simulation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD)*, 2026. doi: 10.1145/3770855.3817497.